

A Reproducible Benchmark for OCR Post-Correction of Low-Resource Arabic-Script Turkic Historical Texts

Radmir

Spring 2026

Abstract

This project studies OCR post-correction for historical Turkic texts written in Arabic script. We formulate the problem as a sequence-to-sequence NLP task: given a noisy OCR-like text line, the system should generate its clean version. We collect a small Arabic-script Turkic corpus, construct a synthetic noisy-clean benchmark, compare identity, rule-based, train-derived character-confusion, and ByT5-small approaches, and evaluate the results with Character Error Rate (CER), Word Error Rate (WER), and Exact Match. On the proposed benchmark, ByT5-small with a 512-token sequence length achieves the best CER and WER among the four evaluated methods. The project code is available at: <https://github.com/Therad445/low-resource-arabic-script-turkic-ocr>.

1 Introduction

Historical Turkic texts written in Arabic script are important for linguistic, cultural and historical research. However, such materials remain difficult to process automatically. The task is low-resource: there are few annotated corpora, orthography is unstable, document quality varies, and standard OCR and NLP tools are usually designed for modern high-resource languages and scripts.

This project focuses on OCR post-correction. We do not attempt to solve full image-based OCR or handwritten text recognition. Instead, we isolate the next NLP stage: correcting an already produced noisy text output. The task is formulated as:

noisy Arabic-script Turkic text $\rightarrow$ clean Arabic-script Turkic text.

The main contributions are:

- a small reproducible benchmark for Arabic-script Turkic OCR post-correction;
- a synthetic OCR-like noise generation protocol;

- a clean-line-level train/validation/test split without leakage;
- comparison of identity, rule-based, train-derived character-confusion and neural sequence-to-sequence approaches;
- evaluation with CER, WER and Exact Match;
- per-sample error analysis of model improvements and degradations.

1.1 Team

Radmir collected and prepared the corpus, constructed the synthetic post-correction dataset, implemented and evaluated the baselines, trained and evaluated the ByT5-small model, performed error analysis, prepared the repository and wrote the final report.

2 Related Work

2.1 Historical OCR and HTR Workflows

OCR for historical documents differs from OCR for modern clean printed text. Historical material often contains unstable typography, damaged scans, non-standard spelling, complex layouts and domain-specific writing conventions. OCR workflows for historical printings therefore usually include data preparation, ground-truth construction, model training, post-processing and manual validation [Reul et al., 2019, Springmann et al., 2018]. These works are important for this project because they show that OCR quality is not only a model problem: it depends on data preparation, segmentation, document layout, transcription conventions and downstream correction.

This project is narrower than a full OCR or HTR workflow. It does not train an image-to-text recognizer. Instead, it isolates the post-correction stage and asks whether a text-to-text model can improve noisy OCR-like output for Arabic-script Turkic historical text.

2.2 Arabic-Script OCR

Arabic-script OCR is challenging because letters change shape depending on their position, the script is cursive, and many characters differ only by small dots or diacritics. These problems become stronger in historical or multilingual contexts, where Arabic, Persian and Turkic orthographic traditions can overlap. Earlier surveys emphasize such script-specific difficulties [Naz et al., 2014]. More recent work on open-source Arabic-script OCR shows that accuracy depends strongly on training data, script coverage, line segmentation, layout analysis and the treatment of error patterns [Kiessling et al., 2024].

This motivates two design choices in the present project. First, the benchmark explicitly keeps Arabic-script characters rather than transliterating the data into Latin script. Second, the dataset analysis reports a character inventory

and a taxonomy of synthetic OCR-like corruptions, because Arabic-script post-correction should not be evaluated only as generic text normalization.

2.3 OCR Post-Correction Approaches

OCR post-correction transforms noisy OCR output into a cleaner textual representation. At the line level, it can be formulated as a sequence-to-sequence problem. Quality is commonly evaluated with edit-distance-based metrics such as CER and WER, which are based on insertions, deletions and substitutions [Levenshtein, 1966].

Several families of approaches exist. Rule-based and normalization-based systems are simple and interpretable, but they can be too aggressive when historical spelling variation is meaningful. Lexicon-based and edit-distance-based methods can help when a reliable vocabulary exists, but they are less suitable for low-resource historical corpora with unstable spelling. Neural post-correction methods learn character-level or sequence-level transformations from noisy-clean pairs. Lyu et al. [Lyu et al., 2021] proposed a neural approach for historical OCR post-hoc correction using character-level modelling and attention. Hamalainen and Hengchen [Hamalainen and Hengchen, 2019] and Duong et al. [Duong et al., 2020] studied unsupervised extraction of parallel data for character-based sequence-to-sequence correction and spelling normalisation in historical corpora.

Compared with these works, this project is smaller in scale but targets a rarer setting: Arabic-script Turkic historical text. The benchmark is also designed to be reproducible within a course project repository.

2.4 Low-Resource and Synthetic-Data Settings

A recurring difficulty in OCR post-correction is the lack of manually aligned noisy-clean data. This is especially severe for historical documents and low-resource languages. Suissa et al. [Suissa et al., 2023] show that OCR correction for historical Hebrew benefits from task-specific training data and that correction quality depends on genre and domain. Naiman et al. [Naiman et al., 2023] use synthetic ground-truth/OCR data for post-correction of historical scientific articles, demonstrating that synthetic noisy-clean pairs can be a practical way to construct large training resources when direct aligned OCR correction data is limited.

The present project follows the same general motivation, but at a smaller pilot scale. Synthetic corruption is not treated as a perfect substitute for real OCR output. Instead, it is used to build a controlled benchmark that can be extended later with real OCR or HTR predictions.

2.5 Transformer and Byte-Level Models

Transformer models [Vaswani et al., 2017] and text-to-text transfer learning [Raffel et al., 2020] provide a general framework for sequence-to-sequence NLP

tasks. In this project, the main neural model is ByT5 [Xue et al., 2022], a byte-level sequence-to-sequence model. Byte-level modelling is useful for noisy and low-resource settings because it avoids dependence on a fixed word or subword vocabulary. This is especially relevant for historical Arabic-script Turkic text, where rare spellings, mixed orthographic conventions and OCR-like distortions can make subword tokenization fragile.

Recent work has also explored large and multimodal language models for historical OCR and post-correction. Greif et al. [Greif et al., 2025] study multimodal LLMs for OCR, OCR post-correction and named entity recognition in historical documents. Such approaches are promising, but they often require large proprietary models or image-based inputs. The present project instead uses a lightweight, reproducible fine-tuning setup on text-level noisy-clean pairs.

2.6 Positioning of This Work

Table 1 summarizes the most relevant previous work and how it relates to this project. The main difference is that this project focuses on a small but explicit Arabic-script Turkic post-correction benchmark, with controlled synthetic noise, leakage-safe splits, baseline comparisons and error analysis.

Work	Setting	Method	Main contribution	Relation to this project
OCR4all [Reul et al., 2019]	Historical printings	OCR workflow	Semi-automatic OCR workflow with ground-truth preparation and model training	Motivates the full historical OCR pipeline; this project focuses only on post-correction
Springmann et al. [Springmann et al., 2018]	Historical printings	OCR and corpus construction	Shows how historical OCR supports diachronic corpora	Provides historical OCR context and the need for downstream correction
Lyu et al. [Lyu et al., 2021]	Historical German corpora	Neural post-correction	Character-level neural correction for historical OCR output	Closest neural post-correction baseline in spirit, but not Arabic-script or Turkic
Hamalainen and Hengchen [Hamalainen and Hengchen, 2019]	Historical corpora	Unsupervised NMT	Automatically extracts parallel data for character-level NMT correction	Alternative to our supervised synthetic noisy-clean benchmark
Duong et al. [Duong et al., 2020]	Finnish historical corpora	Unsupervised seq2seq	Combines OCR correction with spelling normalisation	Shows low-resource post-correction without manual parallel data
Suissa et al. [Suissa et al., 2023]	Historical Hebrew	Neural OCR correction	Studies task-specific training data and genre effects	Supports our focus on script-specific and corpus-specific error modelling
Naiman et al. [Naiman et al., 2023]	Historical scientific articles	Synthetic data	Builds a large synthetic ground-truth OCR post-correction dataset	Justifies synthetic noisy-clean pairs as a practical benchmark strategy
Kiessling et al. [Kiessling et al., 2024]	Arabic-script OCR	Kraken study	Provides Arabic-script OCR evaluation and error microanalysis	Directly motivates Arabic-script error analysis and training-data sensitivity
ByT5 [Xue et al., 2022]	Multilingual NLP	Byte-level Transformer	Avoids subword tokenization and improves robustness to noise	Main model choice for noisy Arabic-script post-correction
Greif et al. [Greif et al., 2025]	Historical documents	Multimodal LLMs	Uses mLLMs for OCR, post-correction and NER	Recent frontier approach; our work is lighter and reproducible without mLLM APIs

Table 1: Comparison with related work on historical OCR, Arabic-script OCR and OCR post-correction.

3 Model Description

The project is structured as a text-level OCR post-correction pipeline rather than a full image-based OCR system. The input to every method is a noisy OCR-like Arabic-script Turkic line, and the output is a corrected line that should match the clean reference. The pipeline is summarized in Table 2.

Component	Input / output	Role in the project
Clean corpus	Historical Arabic-script Turkic text lines	Provides the reference text used as the clean target.
Synthetic noise generator	Clean line $\rightarrow$ noisy OCR-like line	Simulates text-level OCR errors in a controlled and reproducible way.
Leakage-safe splitter	Clean source lines $\rightarrow$ train/valid/test	Ensures that noisy variants of the same clean line do not appear in different splits.
Baselines	Noisy line $\rightarrow$ corrected line	Establish simple non-neural reference points.
ByT5-small	Noisy line $\rightarrow$ predicted clean line	Learns a byte-level sequence-to-sequence post-correction model.
Evaluation	Prediction + clean reference	Computes CER, WER and Exact Match on the held-out test split.
Error analysis	Per-example metric differences	Shows where the model improves the noisy input and where it makes it worse.

Table 2: Main components of the proposed OCR post-correction pipeline.

3.1 Task Formulation

Let x denote a noisy OCR-like line and y denote the corresponding clean historical line. The goal is to learn or define a correction function f such that:

$$\hat{y} = f(x),$$

where $\hat{y}$ should be as close as possible to y according to character-level, word-level and exact-match metrics. This formulation deliberately abstracts away from image processing and focuses on the NLP stage after OCR or HTR has already produced a textual hypothesis.

3.2 Identity Baseline

The identity baseline defines $f(x) = x$. It does not attempt any correction. This baseline is important because it measures the raw difficulty of the noisy

synthetic OCR-like text. If a model does not improve over identity, then it is not useful as a post-correction component. In this project, identity also provides a conservative lower bound: it cannot hallucinate new errors, but it also cannot repair any existing corruption.

3.3 Rule-Based Normalizer

The rule-based normalizer applies simple Arabic-script normalization rules. Its purpose is to test whether hand-written character normalization can already solve a substantial part of the benchmark. This is a meaningful baseline for Arabic-script text because several characters may have visually or historically related forms.

However, this baseline is intentionally limited. Historical Arabic-script Turkic orthography may preserve meaningful variation, and aggressive normalization can destroy distinctions that are important for the clean reference. Therefore, the rule-based baseline is expected to be interpretable but not necessarily strong. Its poor performance in the final results confirms that naive normalization is not sufficient for this benchmark.

3.4 Train-Derived Character Confusion Baseline

To add a data-driven non-neural comparison, I also evaluate a train-derived character confusion baseline. The baseline aligns noisy-clean training pairs with Levenshtein dynamic programming and estimates a conservative mapping from noisy characters to clean characters. A mapping is accepted only when the source character has enough support and the most frequent target character is sufficiently dominant. This prevents the baseline from rewriting rare characters based on weak evidence.

This method is intentionally limited. It can substitute or delete existing characters, but it cannot generate a missing character from context and cannot model long-range dependencies. Its role is therefore diagnostic: it tests whether simple corpus-derived character confusions are enough, or whether a contextual sequence-to-sequence model is needed.

3.5 ByT5-small

The main neural model is ByT5-small [Xue et al., 2022]. It is fine-tuned as an encoder-decoder sequence-to-sequence model. The encoder reads the noisy line, and the decoder generates the corrected line. The training objective is to maximize the likelihood of the clean target sequence conditioned on the noisy input sequence.

ByT5 is appropriate for this task because it operates at the byte level rather than relying on a fixed word or subword vocabulary. This matters for Arabic-script historical text for three reasons. First, rare historical spellings may be poorly represented by standard subword tokenizers. Second, OCR-like noise can create malformed or rare fragments. Third, Arabic-script characters and mixed

orthographic conventions can make vocabulary-based modelling fragile. Byte-level modelling avoids out-of-vocabulary tokens and makes the model directly sensitive to local character distortions.

The final reported configuration uses 512-token source and target lengths and 2 training epochs. The 512-token length is important because the dataset contains relatively long lines, and shorter sequence limits may truncate information needed for correction.

Setting	Value
Model family	ByT5-small
Pretrained checkpoint	<code>google/byt5-small</code>
Task formulation	line-level sequence-to-sequence post-correction
Input	noisy OCR-like text line
Target	clean historical text line
Maximum source length	512 byte-level tokens
Maximum target length	512 byte-level tokens
Training epochs	2
Training batch size	4
Learning rate	5×10^{-5}
Weight decay	0.01
Evaluation strategy	every epoch
Save strategy	every epoch
FP16	false
Generation beams	4
Inference batch size	8
Maximum generated tokens	512
Training pairs	6400
Validation pairs	800
Test pairs	800
Evaluation metrics	CER, WER, Exact Match
Final results file	<code>outputs/postcorrection/final_metrics.csv</code>

Table 3: Final ByT5-small configuration used in the reported experiment.

3.6 Compared Methods

Table 4 summarizes the compared methods. The comparison covers four levels of modelling complexity: no correction, hand-written rule-based correction, data-driven character-level correction and neural sequence-to-sequence correction.

Method	Description	Why it is included
Identity baseline	Returns the noisy input unchanged.	Measures the initial difficulty of the noisy OCR-like text and gives the minimum reference point.
Rule-based normalizer	Applies simple Arabic-script character normalization rules.	Tests whether hand-written normalization is sufficient for this historical material.
Train-derived character confusion baseline	Learns a conservative noisy-character to clean-character mapping from aligned training pairs.	Adds a data-driven non-neural baseline between hand-written rules and neural sequence-to-sequence modelling.
ByT5-small 512 / 2 epochs	Fine-tunes a byte-level encoder-decoder model to map noisy lines to clean lines.	Tests whether a neural sequence-to-sequence model can learn corrections beyond simple normalization and character-level mappings.

Table 4: Compared OCR post-correction methods.

4 Dataset

The dataset is a pilot benchmark for OCR post-correction of Arabic-script Turkic historical text. The clean corpus was collected from an Ottoman Turkish / Arabic-script Turkic Wikisource source. The project does not use document images directly at this stage; instead, it isolates the text-level post-correction problem by generating synthetic OCR-like noisy versions of clean historical lines.

4.1 Corpus Structure and Splits

Each processed CSV file contains four fields:

- **line_id**: identifier of the original clean line;
- **variant_id**: identifier of the synthetic noisy variant;
- **noisy**: synthetically corrupted OCR-like text;
- **clean**: original clean text.

The split is performed by clean source lines, not by individual noisy variants. This prevents leakage: noisy variants of the same original line do not appear in different splits. This is important because otherwise the model could see almost identical versions of the same clean line during training and testing. The

source description is stored in `data/postcorrection/raw/SOURCES.md`. The benchmark is suitable for controlled comparison of post-correction methods, although it does not replace evaluation on real OCR or HTR output.

Split	Clean lines	Noisy-clean pairs	Avg. clean chars	Avg. noisy chars
train	320	6400	113.94	113.51
valid	40	800	112.75	112.24
test	40	800	122.25	121.70

Table 5: Dataset split statistics for the proposed Arabic-script Turkic OCR post-correction benchmark.

4.2 Corpus Length and Character Statistics

Table 6 gives a more detailed view of the corpus. The test split contains 800 noisy-clean pairs derived from 40 unique clean lines. The lines are relatively long for exact sequence-level correction: the average clean test line has more than 120 characters and about 18 words. This helps explain why Exact Match remains low even when CER and WER improve.

Split	Pairs	Clean lines	Avg chars	Median	Range	Avg words
train	6400	320	113.94	115.5	35–180	17.19
valid	800	40	112.75	120.0	41–174	17.15
test	800	40	122.25	122.0	39–178	18.18

Table 6: Length statistics for the processed noisy-clean splits.

The corpus contains a mixed Arabic-script character inventory. Table 7 summarizes the number of characters, while Table 8 reports the most frequent non-space Unicode code points in the clean text. Unicode codes are used to keep the PDF stable and to make the character inventory unambiguous.

Statistic	Clean text	Noisy text
Total characters	917200	913612
Unique characters	114	114

Table 7: Character inventory summary for clean and noisy text.

Unicode code	Count	Share
U+0627	97780	10.66%
U+0631	62620	6.83%
U+0648	61480	6.70%
U+0646	52940	5.77%
U+0644	48840	5.32%
U+06CC	46360	5.05%
U+062F	41660	4.54%
U+064A	34760	3.79%
U+0647	34660	3.78%
U+0645	34440	3.75%
U+062A	32740	3.57%
U+0642	25140	2.74%

Table 8: Most frequent non-space Unicode code points in the clean corpus.

4.3 Synthetic OCR-like Noise

The benchmark uses synthetic noise to simulate common text-level OCR errors. The noise generator is intended to create a controlled post-correction benchmark rather than a fully realistic OCR system. Table 9 summarizes the main types of corruptions represented by the generator and observed in the noisy-clean pairs.

Noise type	Description	Motivation
Character substitution	A character is replaced by another visually or script-related character.	Models OCR confusions between similar Arabic-script shapes.
Character deletion	A character is removed from the clean line.	Models missed glyphs, weak ink, scan noise, or segmentation loss.
Character insertion	An extra character is inserted into the line.	Models spurious OCR glyphs and over-segmentation.
Whitespace split/merge	Spaces may be inserted inside words or removed between tokens.	Models word-boundary errors in cursive OCR.
Arabic/Persian form confusion	Alternative forms of related Arabic-script characters are introduced.	Models mixed Arabic, Persian and Ottoman conventions.

Table 9: Synthetic OCR-like noise taxonomy used in the benchmark.

Table 10 gives an approximate quantitative view of the noise on the test split. Length deltas cannot fully describe OCR errors, because substitutions may preserve length, but they help distinguish deletion/insertion-heavy corruption from purely substitutional corruption.

Statistic on test split	Value
Mean noisy-clean char length delta	-0.554
Median noisy-clean char length delta	0.000
Mean absolute char length delta	1.929
Share of shorter noisy lines	49.38%
Share of equal-length noisy lines	18.75%
Share of longer noisy lines	31.87%
Mean noisy-clean word-count delta	0.601

Table 10: Approximate noisy-clean length-delta statistics on the test split.

4.4 Dataset Examples

Figure 1 shows representative noisy-clean pairs from the test split. The full examples are also stored in the repository under `docs/nlp_final_revision/samples/`. The examples illustrate character substitutions, missing or inserted characters, and word-boundary distortions.

Dataset examples: noisy-clean pairs Example 1: line_id=395, variant_id=0 رېلېبهنلوا خاصېا و امېا دندروس رب یمومع قج آن رلنوب هکدرغونتمو فلتمو رذقوا هیلاغو هیلغف فئاظو یغیدلوی فلک Noisy نوم هلقم راقح هشاب کل هصالخستسا تئیه رېلېبهنلوا خاصېا و امېا دندروس رب یمومع قج آن رلنوب هکدرغونتمو فلتمو رذقوا هیلاغو هیلغف فئاظو یغیدلوا فل Clean نوم هلقم راقح هشاب کل هصالخستسا تئیه
Example 2: line_id=395, variant_id=1 رېلېبهنلوا خاصېا و امېا دندروس رب یمومع قج آن رلنوب هکدرغونتمو فلتمو رذقوی ه یلاغو هیلغف فئاظو یغیدلوا فل Noisy نوم هلقم راقح هشاب کل رهصالخستسا تئیه رېلېبهنلوا خاصېا و امېا دندروس رب یمومع قج آن رلنوب هکدرغونتمو فلتمو رذقوا هیلاغو هیلغف فئاظو یغیدلوا فل Clean نوم هلقم راقح هشاب کل هصالخستسا تئیه
Example 3: line_id=395, variant_id=2 رېلېبهنلوا خاصېا و امېا دندروس رب یمومع قج آن رلنوب هکدرغونتمو فلتمو رذقوا هیلاغو هیلغف فئاظو یغیدلوا فل Noisy نوم هلقم راقح هشاب کل هصالخستسا تئیه رېلېبهنلوا خاصېا و امېا دندروس رب یمومع قج آن رلنوب هکدرغونتمو فلتمو رذقوا هیلاغو هیلغف فئاظو یغیدلوا فل Clean نوم هلقم راقح هشاب کل هصالخستسا تئیه

Figure 1: Examples of synthetic noisy-clean pairs from the test split.

5 Experiments

5.1 Metrics

We use three metrics: Character Error Rate (CER), Word Error Rate (WER), and Exact Match.

CER is computed as:

$$CER = \frac{S + D + I}{N},$$

where S is the number of substitutions, D is the number of deletions, I is the number of insertions, and N is the number of characters in the reference text.

WER is computed analogously at the word level:

$$WER = \frac{S_w + D_w + I_w}{N_w}.$$

Exact Match is the fraction of examples where the predicted line exactly matches the clean reference line. Exact Match is a strict metric for this benchmark because the lines are long and contain historical orthographic variation. Therefore, CER and WER are more informative for partial improvements.

5.2 Experimental Protocol

All methods are evaluated on the same held-out test split. The split is leakage-safe because it is based on clean source lines rather than noisy variants. The identity, rule-based and train-derived character-confusion baselines are evaluated on the test set. The ByT5-small model is fine-tuned on the training split and evaluated on the same held-out test split.

Step	Description
Data preparation	Clean historical lines are converted into synthetic noisy-clean pairs.
Splitting	The split is performed by clean source line to avoid leakage between train, validation and test.
Baseline evaluation	Identity, rule-based and train-derived character-confusion baselines are applied to the test split.
Neural training	ByT5-small is fine-tuned on the training split using noisy lines as inputs and clean lines as targets.
Model selection	The final reported configuration is the 512-token, 2-epoch ByT5-small run stored in the repository artifacts.
Final evaluation	All four methods are evaluated on the same held-out test split of 800 examples.
Error analysis	Per-example CER and WER are compared against the noisy input to count improvements and degradations.

Table 11: Experimental protocol used in the benchmark.

5.3 Reproducibility and Stored Artifacts

The repository stores both the final report and the main intermediate artifacts required to inspect the experiment. Table 12 lists the most important files. The

final metrics are stored in `outputs/postcorrection/final_metrics.csv`. The per-example error analysis is stored under `outputs/postcorrection/error_analysis/`. The reproduction guide is provided in `REPRODUCE.md`.

Artifact	Purpose
<code>data/postcorrection/processed/</code>	Train, validation and test CSV files.
<code>outputs/postcorrection/final_metrics.csv</code>	Main comparison table for all evaluated methods.
<code>outputs/postcorrection/error_analysis/</code>	Per-example improvement/degradation analysis for ByT5.
<code>docs/nlp_final_revision/</code>	Additional audit tables, dataset samples and revision materials.
<code>REPRODUCE.md</code>	Reproduction guide for running the project pipeline.
<code>MODEL_CARD.md</code>	Summary of the model, intended use and limitations.
<code>DATASET_CARD.md</code>	Summary of the dataset, construction and limitations.

Table 12: Important reproducibility artifacts in the repository.

5.4 Baselines and Model Configuration

The benchmark includes three non-neural baselines and one neural model. The identity baseline is a conservative reference point. The rule-based normalizer tests whether simple Arabic-script normalization can repair the synthetic OCR-like errors. The train-derived character confusion baseline tests whether corpus-derived character mappings are enough. The ByT5-small model tests whether a byte-level sequence-to-sequence model can learn context-sensitive corrections.

The final neural configuration is reported in Table 3. Because this is a course-scale pilot project, the experimental grid is intentionally limited. The goal is not to exhaustively tune every architecture, but to establish a reproducible benchmark and demonstrate that a byte-level neural model improves over simple baselines on this dataset.

5.5 Limitations of the Experimental Setup

The main experimental limitation is that the noisy data is synthetic. This makes the benchmark controlled and reproducible, but it does not fully reproduce real OCR or HTR error distributions. Another limitation is the still modest number of evaluated methods. The current comparison includes identity, hand-written rule-based, train-derived character-confusion and neural sequence-to-sequence approaches, but future work should add more architectures, real OCR outputs, multiple random seeds and more extensive hyperparameter search.

6 Results

Table 13 reports the final test-set results. The ByT5-small 512-token model achieves the best CER and WER among the evaluated methods on the proposed benchmark. The non-neural baselines are useful diagnostically: the rule-based normalizer tests hand-written normalization, while the train-derived character confusion baseline tests whether corpus-derived character mappings are sufficient.

Method	CER ↓	WER ↓	Exact Match ↑	N
Identity baseline	0.086005	0.519006	0.001250	800
Rule-based normalizer	0.151932	0.684408	0.000000	800
Train-derived char-confusion baseline	0.082520	0.506189	0.001250	800
ByT5-small 512 / 2 epochs	0.079913	0.368540	0.003750	800

Table 13: Main test results on the proposed benchmark. Lower CER and WER are better; higher Exact Match is better.

6.1 Metric-Level Interpretation

The identity baseline has CER 0.086005 and WER 0.519006. This means that the synthetic OCR-like noise is not catastrophic at the character level, but it still damages word-level structure substantially. The rule-based normalizer increases both CER and WER, which suggests that naive normalization changes many characters that should remain unchanged in the clean historical reference. The train-derived character confusion baseline provides an additional data-driven non-neural comparison and shows whether local character mappings alone can close the gap to a contextual neural model.

ByT5-small improves CER from 0.086005 to 0.079913 and WER from 0.519006 to 0.368540. The WER improvement is stronger than the CER improvement. This suggests that the neural model often restores word-level structure even when it does not exactly recover every character. Exact Match remains low for all methods because the test examples are long: a single remaining character error makes the whole line fail exact matching.

6.2 Why the Rule-Based Baseline Fails

The rule-based normalizer is useful as a diagnostic baseline precisely because it is simple. Its worse performance indicates that the benchmark cannot be solved by blindly collapsing Arabic-script variants. In historical Arabic-script Turkic material, character variation may reflect orthographic convention rather than OCR noise. A safe post-correction model must therefore use context and avoid over-normalization.

6.3 Error Analysis

Table 14 compares each ByT5 prediction against the noisy input. The model improves most examples by both CER and WER, but it is not uniformly safe. Some predictions are worse than the original noisy line.

Criterion	Improved	Unchanged	Worse	Total
CER	682	60	58	800
WER	677	63	60	800

Table 14: Per-sample error analysis for ByT5-small 512 compared with the noisy input.

This failure pattern is important for practical use. The model should be treated as an assistive post-correction component: it can propose useful corrections, but its output should still be inspected by a human or filtered by confidence/error-detection mechanisms. For scholarly transcription, automatic correction without review would be risky.

6.4 Qualitative Examples

In addition to aggregate metrics, we inspected individual examples. Table 15 shows representative improved and degraded cases. The full Arabic-script noisy, predicted and clean strings are provided in `docs/nlp_final/qualitative_examples_full.md`. The examples confirm that ByT5 often repairs local OCR-like distortions, but can also introduce harmful changes.

6.5 SotA Claim on the Proposed Benchmark

Since this benchmark is newly constructed in the project, there is no previously published result on exactly the same dataset and task. Therefore, the SotA claim is intentionally limited: ByT5-small 512 is the best-performing method among the evaluated approaches on the proposed benchmark, including identity, hand-written rule-based and train-derived character-confusion baselines. The project does not claim global state of the art for OCR post-correction in general.

7 Limitations

The current benchmark is intentionally limited. First, the corpus is small and comes from one main source. Second, the noisy-clean pairs are based on synthetic OCR-like corruptions rather than real OCR or HTR outputs. Third, the model is evaluated automatically and has not yet been validated by a domain expert. Finally, some ByT5 predictions are worse than the original noisy input, so the model should be used as an assistive post-correction component rather than a fully automatic scholarly transcription system.

Row	Case	Δ CER	Δ WER	Qualitative interpretation
254	Improved	0.097	0.400	Repairs multiple OCR-like distortions and reduces both character-level and word-level error.
126	Improved	0.093	0.286	Repairs multiple OCR-like distortions and reduces both character-level and word-level error.
582	Improved	0.087	0.364	Repairs multiple OCR-like distortions and reduces both character-level and word-level error.
384	Worse	-3.937	-4.417	Introduces additional changes; this supports the limitation that neural post-correction is not uniformly safe.
392	Worse	-3.810	-4.167	Introduces additional changes; this supports the limitation that neural post-correction is not uniformly safe.

Table 15: Representative qualitative examples for ByT5-small 512. Positive Δ values mean improvement over the noisy input.

8 Conclusion

This project introduced a reproducible pilot benchmark for OCR post-correction of low-resource Arabic-script Turkic historical text. We collected a small clean corpus, generated synthetic OCR-like noisy-clean pairs, created a train/validation/test split without clean-line leakage, and compared identity, rule-based, train-derived character-confusion and ByT5-small approaches.

The ByT5-small 512-token configuration achieved the best CER and WER among the evaluated methods on the proposed benchmark. At the same time, the experiment has important limitations: the corpus is small, the current data comes from one main source, errors are synthetic, and real OCR/HTR outputs should be added in future work. The project therefore represents a reproducible pilot study and a foundation for future work rather than a finished OCR system.

References

- [Duong et al., 2020] Duong, Q., Hamalainen, M., and Hengchen, S. (2020). An unsupervised method for ocr post-correction and spelling normalisation for finnish. *arXiv preprint arXiv:2011.03502*.
- [Greif et al., 2025] Greif, G., Griesshaber, N., and Greif, R. (2025). Multimodal llms for ocr, ocr post-correction, and named entity recognition in historical documents. *arXiv preprint arXiv:2504.00414*.

- [Hamalainen and Hengchen, 2019] Hamalainen, M. and Hengchen, S. (2019). From the past to the future: a fully automatic nmt and word embeddings method for ocr post-correction. *arXiv preprint arXiv:1910.05535*.
- [Kiessling et al., 2024] Kiessling, B., Kurin, G., Miller, M. T., and Smail, K. (2024). Advances and limitations in open source arabic-script ocr: A case study. *arXiv preprint arXiv:2402.10943*.
- [Levenshtein, 1966] Levenshtein, V. I. (1966). Binary codes capable of correcting deletions, insertions, and reversals. *Soviet Physics Doklady*, 10(8):707–710.
- [Lyu et al., 2021] Lyu, L., Koutraki, M., Krickl, M., and Fetahu, B. (2021). Neural ocr post-hoc correction of historical corpora. *arXiv preprint arXiv:2102.00583*.
- [Naiman et al., 2023] Naiman, J. P., Cosillo, M. G., Williams, P. K. G., and Goodman, A. (2023). Large synthetic data from the arxiv for ocr post correction of historic scientific articles. *arXiv preprint arXiv:2309.11549*.
- [Naz et al., 2014] Naz, S., Umar, A. I., Ahmad, R., Siddiqi, I., Ahmed, S. H., Razzak, M. I., and Shafait, F. (2014). A survey of ocr in arabic language: Applications, techniques, and challenges. *Applied Sciences*, 4(3):397–420.
- [Raffel et al., 2020] Raffel, C., Shazeer, N., Roberts, A., Lee, K., Narang, S., Matena, M., Zhou, Y., Li, W., and Liu, P. J. (2020). Exploring the limits of transfer learning with a unified text-to-text transformer. *Journal of Machine Learning Research*, 21(140):1–67.
- [Reul et al., 2019] Reul, C., Christ, D., Hartelt, A., Balbach, N., Wehner, M., Springmann, U., Wick, C., Grundig, C., Büttner, A., and Puppe, F. (2019). Ocr4all—an open-source tool providing a semi-automatic ocr workflow for historical printings. *Applied Sciences*, 9(22):4853.
- [Springmann et al., 2018] Springmann, U., Fink, F., and Schulz, K. U. (2018). Ocr of historical printings with an application to building diachronic corpora: A case study using the ridges herbal corpus. In *Proceedings of the Eleventh International Conference on Language Resources and Evaluation*.
- [Suijsa et al., 2023] Suijsa, O., Elmalech, A., and Zhitomirsky-Geffet, M. (2023). Optimizing the neural network training for ocr error correction of historical hebrew texts. *arXiv preprint arXiv:2307.16220*.
- [Vaswani et al., 2017] Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, L., and Polosukhin, I. (2017). Attention is all you need. In *Advances in Neural Information Processing Systems*.
- [Xue et al., 2022] Xue, L., Barua, A., Constant, N., Al-Rfou, R., Narang, S., Kale, M., Roberts, A., and Raffel, C. (2022). Byt5: Towards a token-free future with pre-trained byte-to-byte models. *Transactions of the Association for Computational Linguistics*, 10:291–306.